Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	30 September 2026
Team ID	SWTID-2026-2365
Project Name	AI FitTrack API – Fitness Tracking API with Google Gemini AI
Maximum Marks	4 Marks

Technical Architecture:

The Deliverable shall include the architectural diagram as below and the information as per the table1 & table 2

AI FitTrack API: RESTful fitness tracking with AI-powered workout recommendations and insights

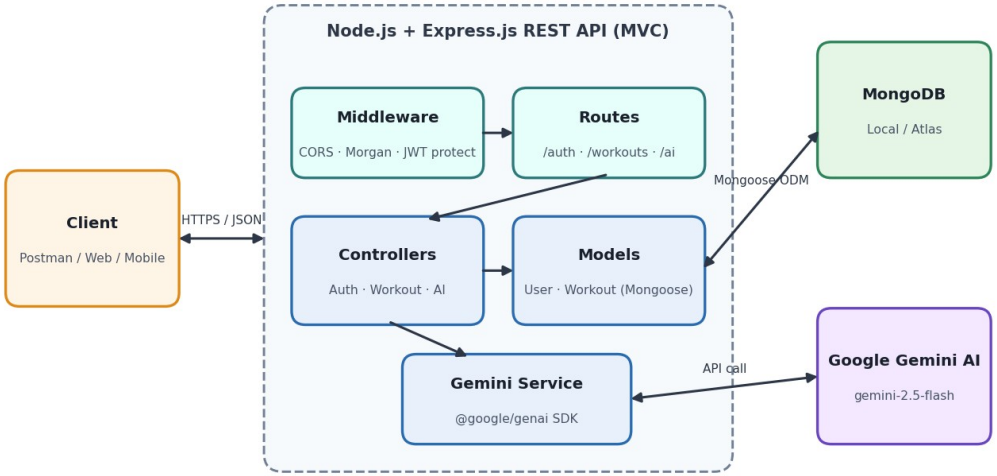


Table-1 : Components & Technologies:

S.No	Component	Description	Technology
1.	User Interface	API consumers (Postman collection, or any web / mobile client) interact with the application through REST endpoints using JSON over HTTP.	Postman, REST / JSON, HTTP
2.	Application Logic-1	Authentication & user management: registration, login and profile, with password hashing and JWT token issuing.	Node.js, Express.js, bcryptjs, jsonwebtoken
3.	Application Logic-2	Workout management: add, view, update and delete workouts, plus search by name, category or date. All data is scoped to the logged-in user.	Node.js, Express.js, Mongoose

4.	Application Logic-3	AI service: builds prompts from user inputs and requests personalised workout recommendations and fitness insights from Gemini.	Google Gemini SDK (@google/genai)
5.	Database	Stores users and workouts as Mongoose schemas (User, Workout) with validation, timestamps and user references.	MongoDB (Mongoose ODM)
6.	Cloud Database	Managed MongoDB database service for cloud deployment (connection through the MONGO_URI setting).	MongoDB Atlas
7.	File Storage	No user file uploads are required. Source code, logs and configuration use the local file system.	Local Filesystem
8.	External API-1	Google Gemini API generates the workout recommendation and the fitness insight text.	Google Gemini API (gemini-2.5-flash)
9.	External API-2	Not used in this application.	N/A
10.	Machine Learning Model	Pre-trained generative AI model used through the Gemini API for personalised fitness guidance. No custom model is trained.	Google Gemini 2.5 Flash (LLM)
11.	Infrastructure (Server / Cloud)	Application deployment on Local System / Cloud Local Server Configuration: Node.js v20+, Express server on port 5000, MongoDB at localhost:27017, nodemon for development. Cloud Server Configuration: MongoDB Atlas cluster; environment variables for PORT, MONGO_URI, JWT_SECRET, GEMINI_API_KEY.	Local (Node.js), MongoDB Atlas (Cloud)

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Node.js runtime with Express.js, Mongoose, CORS, Morgan (request logging), dotenv and nodemon.	Node.js, Express.js, Mongoose
2.	Security Implementations	JWT authentication (30-day expiry) guards all workout and AI routes; passwords are hashed with bcrypt; users can access only their own workouts; secrets are kept in a git-ignored .env file; schema and input validation.	JWT, bcryptjs, dotenv
3.	Scalable Architecture	Layered MVC design (routes, controllers, models, services). Stateless JWT authentication lets several API instances run side by side, and the AI logic is isolated in its own service layer.	Express.js (MVC), stateless REST
4.	Availability	Stateless API instances can be placed behind a load balancer; MongoDB Atlas provides replicated storage; centralised error handling and unhandled-rejection handling keep failures controlled.	MongoDB Atlas, Express error middleware
5.	Performance	Non-blocking async I/O for database and AI calls; user-scoped, date-sorted MongoDB queries; short, concise prompts and the fast Gemini 2.5 Flash model keep AI responses quick.	Node.js async/await, Mongoose, Gemini 2.5 Flash

References:

https://c4model.com/

https://developer.ibm.com/patterns/online-order-processing-system-during-pandemic/

https://www.ibm.com/cloud/architecture

https://aws.amazon.com/architecture

https://medium.com/the-internal-startup/how-to-draw-useful-technical-architecture-diagrams-2d20c9fda90d